

Aufgabe 1. Implementieren Sie das Jacobi-Verfahren zur gleichzeitigen Bestimmung aller Eigenwerte und Eigenvektoren einer symmetrischen Matrix. Beachten Sie dabei, dass zur Berechnung der Matrizen Q_k als Vorzeichenfunktion 

$$\text{sign}(t) = \begin{cases} 1 & t \geq 0 \\ -1 & t < 0 \end{cases}$$

benutzt werden muss, d.h. $\text{sign}(0) = 1$ und *nicht* $\text{sign}(0) = 0$ wie in der `numpy`-Implementierung von `sign`.

Testen Sie Ihr Programm zunächst an einfachen Beispielen. Berechnen Sie dann die Eigenwerte und Eigenvektoren der Block-Tridiagonal-Matrix

$$A = \begin{pmatrix} B & -I & & & \\ -I & B & -I & & \\ & \ddots & \ddots & \ddots & \\ & & -I & B & -I \\ & & & -I & B \end{pmatrix} \in \mathbb{R}^{m^2 \times m^2}, \quad B = \begin{pmatrix} 4 & -1 & & & \\ -1 & 4 & -1 & & \\ & \ddots & \ddots & \ddots & \\ & & -1 & 4 & -1 \\ & & & -1 & 4 \end{pmatrix} \in \mathbb{R}^{m \times m}.$$

für $m = 10$. Iterieren Sie so lange, bis die Quadratsumme der Nebendiagonalelemente kleiner 10^{-3} ist.

Vergleichen Sie die Näherungen des Jacobi-Verfahrens mit den exakten Eigenwerten (siehe unten).

Da A ein einfaches physikalisches Modell für die Verformung einer quadratischen elastischen Membran ist, entsprechen die Eigenwerte den Resonanzfrequenzen der Membran bzw. die Eigenvektoren den Eigenschwingungsformen. Stellen Sie die Eigenschwingungsformen zu den 4 kleinsten Eigenwerten graphisch dar.

In der Datei `membran_lib.py` finden Sie folgende Hilfsfunktionen:

Ablock(m): erzeugt A als dünn besetzte Matrix im CSR Format (bei Matrix-Vektor-Multiplikation Klassenmethode `A.dot()` benutzen)

ew_exakt(m): berechnet die exakten Eigenwerte

plotev(v): stellt die Eigenschwingungsform zum Eigenvektor v graphisch dar

animev(v): animiert die Eigenschwingung zum Eigenvektor v (Aufruf: `animev(v)()`)

Aufgabe 2. Transformieren Sie die Matrix 

$$\begin{pmatrix} 10 & 15 & 20 \\ 15 & -50 & 25 \\ 20 & 25 & -75 \end{pmatrix}$$

mit Hilfe von Givens-Rotationen (von Hand) auf Hessenberg-Form.